

Manuel d'utilisation — metrics-app

Plateforme de monitoring Kubernetes avec prédiction LLM

1. Introduction

Projet : metrics-app **Auteurs :** Romain Barthélémy, Jean-Baptiste RAFFI **Classe :** ETN A 5 **Cours :** Projet R&D **Date :** Mai 2026

Présentation du projet

metrics-app est une plateforme de monitoring d'un cluster Kubernetes à trois nœuds. Elle collecte en continu les métriques CPU et RAM de chaque nœud via Prometheus, les persiste dans une base PostgreSQL, et les expose dans un dashboard web temps réel.

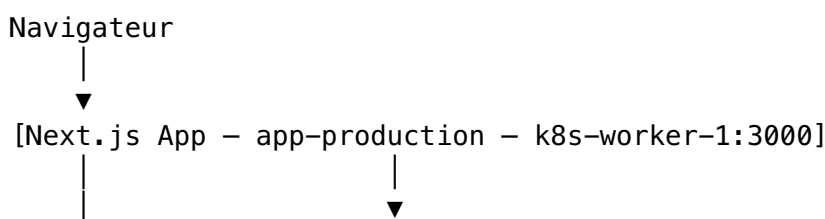
L'application intègre deux fonctionnalités avancées :

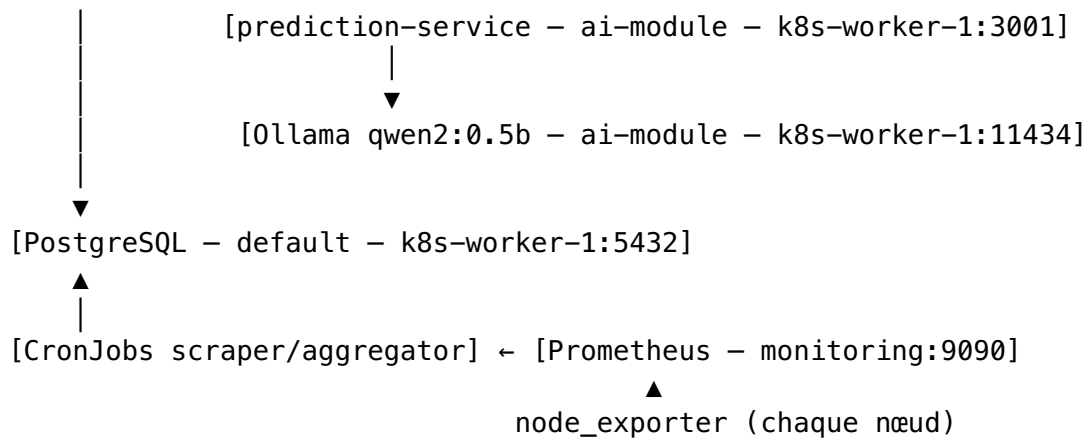
- **Réservation de ressources** : un opérateur peut allouer des quotas CPU/RAM sur un namespace Kubernetes directement depuis l'interface, avec libération automatique à expiration.
- **Prédiction LLM** : un module IA utilise un modèle de langage léger (qwen2:0.5b via Ollama) pour prédire l'évolution de la charge CPU et RAM sur les 30 prochaines minutes, et évaluer le risque de surcharge (low / medium / high).

2. Architecture et choix techniques

2.1 Vue d'ensemble de l'architecture

Le système est composé de six composants déployés sur un cluster Kubernetes à trois nœuds (k8s-master, k8s-worker-1, k8s-worker-2) :





Composant	Namespace	Nœud K8s	Port	Rôle
Next.js App	app-production	k8s-worker-1	3000	Interface web, API REST, orchestration
prediction-service	ai-module	k8s-worker-1	3001	Prédiction CPU/RAM via prompts LLM
Ollama	ai-module	k8s-worker-1	11434	Moteur d'inférence LLM (qwen2:0.5b)
PostgreSQL	default	k8s-worker-1	5432	Stockage métriques, réservations, forecasts
Prometheus	monitoring	—	9090	Collecte métriques node_exporter
Grafana	monitoring	—	3000	Visualisation Prometheus (optionnel)

Flux de données principal : 1. node_exporter expose les métriques brutes de chaque nœud sur le port 9100 2. Prometheus scrape node_exporter toutes les 30 secondes 3. Le CronJob scraper interroge Prometheus et insère les données dans PostgreSQL 4. Next.js lit PostgreSQL pour afficher les métriques en temps réel 5. Pour la prédiction, Next.js délègue au prediction-service qui interroge directement Prometheus et Ollama

2.2 Le système de prédiction — fonctionnement détaillé

Pipeline d'une prédiction forecast

Quand l'utilisateur lance une prédiction pour un nœud donné, voici ce qui se passe :

1. **Requête client** → **Next.js** Le navigateur envoie POST /api/forecast avec :

```
{ "node": "k8s-worker-1", "horizon_minutes": 30,
  "step_minutes": 5 }
```

2. **Next.js** → **prediction-service** Next.js valide la requête, vérifie que le nœud existe en base, puis appelle POST http://prediction-service.ai-module.svc.cluster.local:3001/forecast avec les mêmes paramètres.

3. **prediction-service** → **Prometheus** Le service interroge Prometheus pour récupérer l'historique réel CPU et RAM sur les 30 dernières minutes avec un pas de 5 minutes (6 points). Requêtes PromQL utilisées :

```
CPU : round(100 -  
(avg(rate(node_cpu_seconds_total{mode="idle",instance="192.168.  
[5m])) * 100), 0.1)  
RAM : round((1 -  
node_memory_MemAvailable_bytes{instance="192.168.10.243:9100"}  
node_memory_MemTotal_bytes{instance="192.168.10.243:9100"}) *  
100, 0.1)
```

4. **Prédiction auto-régressive (6 steps pour un horizon de 30min/5min)** Le service génère les prédictions step par step. À chaque itération :
- Il construit un prompt incluant l'historique connu
 - Il envoie ce prompt à Ollama (qwen2:0.5b)
 - Il extrait les valeurs CPU/RAM prédites de la réponse JSON du LLM
 - Ces nouvelles valeurs deviennent l'entrée du step suivant (auto-régression)
5. **Calcul du risque** Une fois les 6 steps générés, le service calcule :
- `cpu_peak / ram_peak` : valeurs maximales prédites
 - Risque : high si `cpu_peak > 90%` ou `ram_peak > 95%`, medium si `cpu_peak > 75%` ou `ram_peak > 85%`, low sinon.
6. **Retour et sauvegarde** Le résultat est retourné à Next.js, sauvegardé en base, puis envoyé au client pour affichage sur le graphique ForecastChart.

Aperçu des prompts (voir Annexe 6.1 pour les versions complètes)

- **Prompt /predict** : donne l'historique CPU et RAM, demande les valeurs suivantes en JSON `{"cpu_percent": X, "ram_percent": Y}`
- **Prompt /forecast step** : donne l'historique détaillé avec timestamps relatifs, demande d'identifier la tendance puis prédire le prochain step en JSON

2.3 Pourquoi un LLM plutôt qu'un réseau de neurones classique ?

L'alternative classique : LSTM / RNN

Pour prédire des séries temporelles de métriques, l'approche classique serait un réseau de neurones récurrents de type LSTM (Long Short-Term Memory). Ce type de modèle est conçu précisément pour capturer les dépendances temporelles dans des données séquentielles (charge CPU qui monte chaque matin à 9h, par exemple).

Pourquoi nous avons choisi un LLM

Critère	LSTM/RNN	LLM (qwen2:0.5b)
Données d'entraînement	Des semaines/mois de métriques labellisées nécessaires	Aucune — le prompt suffit
Mise en service		

Critère	LSTM/RNN	LLM (qwen2:0.5b)
	Pipeline : collecte → entraînement → évaluation → déploiement	Immédiat, modèle pré-entraîné
Flexibilité	Changer le comportement = réentraîner	Changer le comportement = modifier le prompt
Explication	Boîte noire numérique	Peut générer une explication en langage naturel
Précision sur patterns cycliques	Excellente (après entraînement)	Correcte, pas optimale
Latence	< 10 ms par inférence	1-3 s par step (6-18 s pour 30 min)
Ressources	Léger à l'inférence	~350 MB de modèle, 2 GB RAM minimum

Justification du choix dans notre contexte

Dans le contexte de ce projet académique, le LLM présente trois avantages décisifs :

1. **Aucun bootstrap de données** : un LSTM nécessite d'accumuler suffisamment de métriques historiques avant de pouvoir être entraîné. Avec un LLM, la prédiction fonctionne dès le premier jour.
2. **Raisonnement explicable** : le modèle identifie la tendance en une phrase avant de prédire (ex: "CPU steadily increasing, likely to continue rising"). Cette transparence est précieuse pour un outil d'aide à la décision opérationnelle.
3. **Déploiement simplifié** : Ollama + qwen2:0.5b se déploie en un seul pod K8s, sans infrastructure MLOps dédiée (MLflow, feature store, serving GPU, etc.).

Limite principale

La précision sur des patterns très réguliers (charge de pointe quotidienne identique) serait meilleure avec un LSTM entraîné. Le LLM raisonne sur la tendance récente mais ne "mémorise" pas les cycles long-terme de l'infrastructure.

2.4 Scénario de charge de pointe

Ce scénario illustre l'utilité concrète du module de prédiction lors d'une montée en charge prévisible (arrivée des utilisateurs le matin).

Contexte

k8s-worker-1 héberge l'application Next.js, le prediction-service et Ollama. À 8h50, un opérateur consulte le dashboard et observe la tendance suivante :

Heure	CPU (%)	RAM (%)
08:20	22	44

Heure	CPU (%)	RAM (%)
08:25	28	45
08:30	35	47
08:35	48	50
08:40	61	53
08:45	70	56

La courbe CPU monte régulièrement (+8 à +13 points toutes les 5 minutes).

Forecast lancé à 8h50 (horizon 30 min, step 5 min)

L'opérateur lance un forecast sur k8s-worker-1. Le LLM identifie la tendance : > "CPU steadily increasing due to morning load spike, likely to continue rising."

Prédiction générée :

Heure prévue	CPU prédit (%)	RAM prédite (%)	Risque
08:55	77	59	medium
09:00	82	61	medium
09:05	86	63	medium → high
09:10	89	65	high
09:15	92	67	high
09:20	94	69	high — "Intervention requis immédiatement"

Actions recommandées affichées dans le dashboard

- Badge rouge sur la carte de k8s-worker-1
- Recommandation : "Intervention requise immédiatement"
- Options disponibles via l'interface :
 1. **Libérer des réservations** : aller dans Réservations et libérer les quotas non critiques sur k8s-worker-1
 2. **Réduire la charge** : migrer manuellement des déploiements secondaires vers k8s-worker-2 via `kubectl scale` ou en modifiant le `nodeSelector`

Ce que ce scénario démontre

Le module de prédiction donne à l'opérateur une fenêtre d'action de 15 à 20 minutes avant la saturation, permettant une intervention proactive plutôt que réactive.

3. Prérequis et installation

3.1 Infrastructure matérielle

Le projet requiert trois machines Linux (physiques ou VMs) avec Ubuntu 22.04 LTS.

Machine	IP	CPU	RAM	Disque	Rôle K8s
k8s-master	192.168.10.213	2 vCPU	4 GB	30 GB	Control plane
k8s-worker-1	192.168.10.243	4 vCPU	8 GB	50 GB	Worker principal (Ollama, app, DB)
k8s-worker-2	192.168.10.126	2 vCPU	4 GB	30 GB	Worker secondaire

Note : k8s-worker-1 nécessite 8 GB de RAM minimum pour faire tourner simultanément Ollama (2-4 GB), PostgreSQL (512 MB), le prediction-service (256 MB) et Next.js (512 MB).

3.2 Logiciels requis

À installer sur **chaque machine** :

```
# Docker Engine
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# kubeadm, kubelet, kubectl (version 1.28)
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.28/deb/
    Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/
    kubernetes-apt-keyring.gpg
echo
    'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-
    keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.28/
    deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo systemctl enable kubelet
```

À installer sur **k8s-master uniquement** :

```
# Helm 3
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-
    helm-3 | bash
```

```
# git
sudo apt-get install -y git
```

3.3 Configuration réseau

Sur **chaque machine**, ajouter dans /etc/hosts :

```
192.168.10.213 k8s-master
192.168.10.243 k8s-worker-1
192.168.10.126 k8s-worker-2
192.168.10.213 metrics.local
```

Désactiver le swap (requis par Kubernetes) :

```
sudo swapoff -a
sudo sed -i '/ swap / s/^#/' /etc/fstab
```

Initialiser le cluster (sur k8s-master) :

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
# Installer Flannel (réseau pods)
kubectl apply -f https://raw.githubusercontent.com/flannel-io/
    flannel/master/Documentation/kube-flannel.yml
```

Joindre les workers (commande affichée par kubeadm init, à exécuter sur chaque worker) :

```
sudo kubeadm join 192.168.10.213:6443 --token <token> --discovery-
    token-ca-cert-hash sha256:<hash>
```

Installer nginx Ingress Controller :

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/
    ingress-nginx/controller-v1.8.2/deploy/static/provider/
    baremetal/deploy.yaml
```

4. Déploiement complet

4.1 Récupération du code source

Sur k8s-master :

```
git clone https://github.com/Blipblooop/metrics-app.git /opt/
    metrics-app
cd /opt/metrics-app
```

4.2 Namespaces Kubernetes

```
kubectl create namespace app-production
kubectl create namespace ai-module
```

Vérification :

```
kubectl get namespaces
# Doit afficher : app-production, ai-module, default, monitoring
# (après Prometheus)
```

4.3 Base de données PostgreSQL

```
kubectl apply -f k8s/postgres/postgres-storage.yaml
kubectl apply -f k8s/postgres/postgres-statefulset.yaml
kubectl apply -f k8s/postgres/postgres-service.yaml
```

Attendre que le pod soit prêt :

```
kubectl wait --for=condition=ready pod -l app=postgres -n default
--timeout=120s
```

Vérification :

```
kubectl get pod -l app=postgres -n default
# STATUS doit être Running
```

4.4 Prometheus et Grafana

```
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-
stack \
  --namespace monitoring \
  --create-namespace \
  --set grafana.adminPassword=admin
```

Appliquer le ServiceMonitor de l'application :

```
kubectl apply -f k8s/servicemonitor.yaml
kubectl apply -f k8s/grafana-datasource-postgres.yaml
```

Vérification :

```
kubectl get pods -n monitoring
# Tous les pods doivent être Running (peut prendre 2-3 minutes)
```

4.5 Application Next.js

Sur k8s-master, construire l'image et la transférer sur k8s-worker-1 :


```
cd /opt/metrics-app
chmod +x scripts/deploy.sh
./scripts/deploy.sh
```

Ce script effectue les opérations suivantes : 1. `git pull` pour récupérer la dernière version 2. `docker build` de l'image Next.js 3. `docker save + scp` vers `k8s-worker-1` 4. Redémarrage du déploiement K8s

Appliquer les manifests K8s (première fois uniquement) :

```
kubectl apply -f k8s/nextjs-app.yaml
```

Vérification :

```
kubectl get pods -n app-production
kubectl get ingress -n app-production
# L'ingress metrics-app-ingress doit apparaître avec l'host
  metrics.local
```

4.6 Ollama (moteur LLM)

Créer le répertoire de stockage sur `k8s-worker-1` :

```
ssh romain@192.168.10.243 "sudo mkdir -p /data/ollama-models &&
  sudo chmod 777 /data/ollama-models"
```

Déployer Ollama sur K8s :

```
kubectl apply -f k8s/ollama-stack.yaml
kubectl wait --for=condition=ready pod -l app=ollama -n ai-module
  --timeout=300s
```

Télécharger le modèle `qwen2:0.5b` dans le pod Ollama :

```
OLLAMA_POD=$(kubectl get pod -n ai-module -l app=ollama -o
  jsonpath='{.items[0].metadata.name}')
kubectl exec -n ai-module $OLLAMA_POD -- ollama pull qwen2:0.5b
```

Cette commande peut prendre 5-10 minutes selon la connexion réseau (modèle ~350 MB).

Vérification :

```
kubectl exec -n ai-module $OLLAMA_POD -- ollama list
# Doit afficher : qwen2:0.5b
```

4.7 Prediction-service

```
kubectl apply -f k8s/prediction-service.yaml
kubectl wait --for=condition=ready pod -l app=prediction-service
  -n ai-module --timeout=120s
```

Vérification du service :

```
kubectl exec -n ai-module deployment/prediction-service -- \
  wget -qO- http://localhost:3001/health
# Doit retourner : {"status":"ok","model":"qwen2:0.5b",...}
```

4.8 CronJobs

```
kubectl apply -f k8s/cronjob.yaml
kubectl apply -f k8s/cronjob-aggregator.yaml
kubectl apply -f k8s/cronjob-auto-release.yaml
kubectl apply -f k8s/cronjob-auto-reserve.yaml
kubectl apply -f k8s/cronjob-cleanup.yaml
kubectl apply -f k8s/cronjob-events.yaml
```

Vérification :

```
kubectl get cronjobs -n app-production
# Tous les CronJobs doivent être listés avec SCHEDULE affiché
```

4.9 Vérification finale

Accéder à l'application dans un navigateur :

`http://metrics.local`

Si `metrics.local` ne résout pas, vérifier que `/etc/hosts` contient bien la ligne `192.168.10.213 metrics.local` sur la machine cliente.

Tester la santé de tous les composants :

```
# Tous les pods en Running
kubectl get pods --all-namespaces

# Forcer le scraper immédiatement (sans attendre le cron)
kubectl create job --from=cronjob/metrics-scraper manual-scraper-
  test -n app-production

# Vérifier les logs de l'application
kubectl logs -n app-production deployment/metrics-app --tail=20
```

5. Guide utilisateur

5.1 Connexion

Ouvrir un navigateur et accéder à : `http://metrics.local`

La page de login s'affiche. Saisir les identifiants par défaut : - **Identifiant** : admin -
Mot de passe : admin1234

En production, ces identifiants sont à modifier via les variables d'environnement ADMIN_USERNAME et ADMIN_PASSWORD dans k8s/nextjs-app.yaml.

5.2 Dashboard principal

Après connexion, le dashboard affiche :

Cartes de nœuds (NodeCard) Une carte par nœud du cluster (k8s-master, k8s-worker-1, k8s-worker-2), affichant : - CPU actuel en pourcentage avec jauge visuelle - RAM actuelle en pourcentage avec jauge visuelle - Statut du nœud (badge vert/orange/rouge selon la charge) - Alerte si une réservation est active sur ce nœud

Graphique CPU/RAM (CpuRamChart) Sous les cartes, un graphique de courbes affiche l'évolution historique des métriques (1 heure par défaut) pour le nœud sélectionné.

Rafraîchissement automatique Le dashboard se rafraîchit toutes les 30 secondes pour afficher les métriques à jour.

5.3 Panel de prédiction

Le panel de prédiction est accessible depuis le dashboard principal.

Lancer un forecast : 1. Sélectionner le nœud à analyser dans le menu déroulant 2. Laisser les paramètres par défaut : horizon = 30 minutes, step = 5 minutes 3. Cliquer sur **“Lancer la prédiction”** 4. Patienter 10-20 secondes (le LLM génère 6 steps successifs)

Lire les résultats : - Le graphique ForecastChart affiche la courbe de prédiction CPU et RAM sur les 30 prochaines minutes - Un badge indique le niveau de risque global : - **Vert (low)** : charge prévue maîtrisée, aucune action requise - **Orange (medium)** : charge élevée prévue, surveillance recommandée - **Rouge (high)** : saturation prévue, intervention requise

Drawer LLM (détail du raisonnement) : En cliquant sur **“Voir le raisonnement du LLM”**, un panneau latéral s'ouvre et affiche pour chaque step prédit : - Le prompt envoyé au modèle - La réponse brute du LLM (incluant son analyse de tendance) - Les valeurs CPU/RAM extraites

Cette fonctionnalité permet de comprendre et auditer les décisions du modèle.

5.4 Réservation manuelle de ressources

La section Réservations permet d'allouer des quotas CPU/RAM sur un namespace Kubernetes.

Créer une réservation : 1. Choisir le namespace cible dans la liste déroulante 2. Saisir les ressources à réserver (ex : CPU = 500m, RAM = 256Mi) 3. Définir la durée de la réservation (en minutes) 4. Cliquer sur **“Réserver”**

L'application crée un ResourceQuota et un LimitRange sur ce namespace via l'API Kubernetes.

Libérer une réservation : Cliquer sur le bouton **“Libérer”** à côté de la réservation active dans la liste. Les quotas K8s associés sont supprimés immédiatement.

Libération automatique : Un CronJob vérifie toutes les 5 minutes les réservations expirées et les libère automatiquement sans intervention manuelle.

5.5 Événements et alertes Kubernetes

La section Événements affiche en temps réel les événements K8s collectés par le CronJob cronjob-events : - Pods en état Pending ou CrashLoopBackOff - Nœuds NotReady - Échecs de scheduling

Un badge rouge dans le header indique le nombre d'alertes actives non acquittées.

6. Annexes

6.1 Prompts IA utilisés

Ces prompts sont envoyés au modèle Ollama qwen2:0.5b par le prediction-service.

Prompt /predict (prédiction simple)

Source : prediction-service-fix/index.js, ligne 125

```
Node "${node}". CPU: ${cpuHistory.join(', ')}%. RAM: ${ramHistory.join(', ')}%.  
Predict next values. Reply ONLY with valid JSON:  
{"cpu_percent":50,"ram_percent":60}
```

Exemple concret pour k8s-worker-1 :

```
Node "k8s-worker-1". CPU: 22.0, 28.5, 35.1, 48.3, 61.0, 70.2%.  
RAM: 44.1, 45.0, 47.2, 50.1, 53.3, 56.0%.  
Predict next values. Reply ONLY with valid JSON:  
{"cpu_percent":50,"ram_percent":60}
```

Réponse attendue du LLM :

```
{"cpu_percent": 76, "ram_percent": 58}
```

Prompt /forecast (un prompt par step, approche auto-régressive)

Source : prediction-service-fix/index.js, fonction buildStepPrompt, ligne 69

Node "\${node}". CPU history: \${cpuStr}. RAM history: \${ramStr}.
Identify the trend in one sentence, then reply with JSON
{"cpu_percent": <0-100>, "ram_percent": <0-100>}

Exemple au step 1 :

Node "k8s-worker-1".
CPU history: t-30min: 22.0%, t-25min: 28.5%, t-20min: 35.1%,
t-15min: 48.3%, t-10min: 61.0%, t-5min: 70.2%.
RAM history: t-30min: 44.1%, t-25min: 45.0%, t-20min: 47.2%,
t-15min: 50.1%, t-10min: 53.3%, t-5min: 56.0%.
Identify the trend in one sentence, then reply with JSON
{"cpu_percent": <0-100>, "ram_percent": <0-100>}

Réponse typique du LLM :

CPU is steadily increasing due to a morning load spike, likely to
continue rising.
{"cpu_percent": 77, "ram_percent": 59}

Paramètres d'inférence : temperature: 0.1, num_predict: 256

temperature: 0.1 rend le modèle quasi-déterministe pour limiter les variations
imprévisibles.

6.2 Variables d'environnement

Next.js App (k8s/nextjs-app.yaml)

Variable	Valeur par défaut	Description
DATABASE_URL	postgresql:// metrics_user:metrics_password@postgres:5432/ k8s_metrics	Connexion PostgreSQL
JWT_SECRET	change-me-in-production	Secret pour les tokens JWT
ADMIN_USERNAME	admin	Identifiant administrateur
ADMIN_PASSWORD	admin1234	Mot de passe administrateur
PREDICTION_SERVICE_URL	http://prediction-service.ai- module.svc.cluster.local:3001	URL prediction- service
PROMETHEUS_URL	http://monitoring-kube-prometheus- prometheus.monitoring.svc:9090	URL Prometheus

prediction-service (k8s/prediction-service.yaml)

Variable	Valeur par défaut	Description
OLLAMA_URL	http://ollama-service:11434	URL Ollama
OLLAMA_MODEL	qwen2:0.5b	Modèle à utiliser
PROMETHEUS_URL	http://monitoring-kube-prometheus-prometheus.monitoring.svc.cluster.local:9090	URL Prometheus

6.3 Commandes de diagnostic

État de tous les pods

```
kubectl get pods --all-namespaces
```

Logs de l'application Next.js

```
kubectl logs -n app-production deployment/metrics-app --tail=50
```

Logs du prediction-service

```
kubectl logs -n ai-module deployment/prediction-service --tail=50
```

Logs Ollama

```
kubectl logs -n ai-module deployment/ollama --tail=20
```

Tester Ollama directement

```
kubectl exec -n ai-module deployment/ollama -- \
  ollama run qwen2:0.5b "Reply with JSON: {\\"cpu_percent\\": 50}"
```

Vérifier la santé du prediction-service

```
kubectl exec -n ai-module deployment/prediction-service -- \
  wget -qO- http://localhost:3001/health
```

Forcer le scraper immédiatement (sans attendre le cron)

```
kubectl create job --from=cronjob/metrics-scraper manual-scraper-
test -n app-production
```

Vérifier les réservations actives en base

```
kubectl exec -n default statefulset/postgres -- \
  psql -U metrics_user -d k8s_metrics -c "SELECT * FROM
  reservations WHERE status='active';"
```

Redéployer l'app après une modification

```
cd /opt/metrics-app && ./scripts/deploy.sh
```